INSTITUTO FEDERAL Ceará Campus Tauá	**Curso de Análise e Desenvolvimento de Sistemas**	
	Disciplina: Padrões de Projeto de Software	**Data:** ___/___/_____
	Professor: Samuel Soares	
Aluno:		

Implementação 4

1) OBSERVER

Sistema de Alertas Dinâmicos no E-commerce

Contexto do Problema: Imagine o back-end de uma grande plataforma de e-commerce (como Amazon ou Mercado Livre). O preço de um produto não é estático; ele flutua diariamente com base na demanda, estoque e ações dos concorrentes. Toda vez que o preço de um Produto cai ou sobe, vários setores da empresa precisam reagir imediatamente.

O Problema Arquitetural: suponha que a classe Produto tenha um método setPreco(double valor). Dentro desse método, o programador anterior colocou chamadas diretas para outros sistemas:

- Chamou a API do Marketing para enviar notificações aos usuários: "O produto que você quer baixou de preço!".
- Chamou a API de Vendas para atualizar o dashboard de projeção de lucros.
- Chamou a API de Logística para recalcular o seguro do frete de alto valor.

O código está altamente acoplado. Se o setor de "Customer Success" também quiser ser avisado das mudanças de preço, você terá que abrir a classe Produto e adicionar mais um if ou chamada de método. Isso viola o Princípio de Responsabilidade Única (SRP) e o Princípio Aberto/Fechado (OCP) do SOLID.

Refatore este cenário utilizando o padrão Observer:

1. A classe Produto deve ser o *Publisher*. Ela não deve conhecer nada sobre Marketing, Vendas ou Logística.
2. Crie uma interface ObservadorDePreco com um método atualizar(Produto p).
3. Implemente as classes DepartamentoMarketing, DepartamentoVendas e DepartamentoLogistica assinando (se inscrevendo) nas atualizações do Produto.
4. Demonstre na classe principal (Main) como adicionar ou remover um observador em tempo de execução sem alterar o código do Produto.

2) STATE

Máquina de Estados de um Pedido

Contexto do Problema: Sistemas de compras online dependem estritamente do ciclo de vida de um pedido. Um pedido nasce como Novo, passa para AguardandoPagamento, vai para EmSeparacao, depois Enviado e, finalmente, Entregue.

O Problema Arquitetural: A classe Pedido possui métodos como cancelar(), pagar() e despachar(). Atualmente, cada um desses métodos é um pesadelo de manutenção cheio de blocos switch/case ou if/else baseados em uma variável de texto String status. Por exemplo, ao chamar cancelar():

- Se o status for Novo ou AguardandoPagamento, o cancelamento é feito na hora.
- Se o status for EmSeparacao, exibe um aviso alertando o estoque para parar o pacote.
- Se for Enviado, o sistema não pode cancelar e deve obrigar o cliente a iniciar um processo de "Logística Reversa" (devolução).

Se precisarmos adicionar um novo status chamado EmAnaliseDeFraude, o desenvolvedor terá que caçar e alterar todos os métodos dentro da classe Pedido para incluir esse novo cenário.

Utilize o padrão State para eliminar os if/else baseados em status:

1. Crie uma interface abstrata (ou classe abstrata) chamada EstadoPedido contendo os métodos de ação (cancelar(), pagar(), despachar()).
2. Implemente classes concretas para cada estado (EstadoAguardandoPagamento, EstadoEnviado, etc.). Cada classe deve ter a regra de negócio específica do que acontece se o usuário tentar agir *naquele momento*.
3. A classe Pedido deve apenas manter uma referência para o seu estado atual e delegar as ações para este objeto de estado. Mostre como as transições de estado ocorrem internamente (um estado criando e passando o bastão para o próximo).

3) STRATEGY

Motor de Regras de Frete e Descontos

Contexto do Problema: Na etapa de checkout de uma loja virtual, o usuário precisa escolher como deseja receber sua mercadoria. O valor final do pedido depende totalmente dessa escolha.

O Problema Arquitetural: A loja possui uma classe CalculadoraDeFrete. Dentro dela, existe um método calcularValorFinal(Carrinho carrinho, String tipoFrete). Atualmente, as regras são:

- *PAC:* R$ 15,00 fixos + R$ 1,00 por quilo.
- *Sedex:* R$ 30,00 fixos + R$ 2,50 por quilo.
- *Motoboy (Local):* R$ 20,00 fixos, ignorando o peso.
- *Retirada na Loja:* R$ 0,00.

O código atual está estruturado em uma grande cadeia de `if (tipoFrete.equals("SEDEX")) { ... } else if (...)`.

O problema é que transportadoras mudam suas regras constantemente e a loja frequentemente faz parcerias com novas empresas de entrega. Cada nova transportadora infla essa classe, dificultando testes isolados e aumentando o risco de quebrar regras antigas ao mexer no código.

Aplique o padrão Strategy para encapsular esses algoritmos de cálculo:

1. Crie uma interface EstrategiaDeFrete com o método calcular(Carrinho carrinho).
2. Extraia cada regra de frete (PAC, Sedex, Motoboy) para sua própria classe isolada que implementa a interface.
3. A classe Pedido (ou Checkout) deve receber a estratégia escolhida pelo usuário de forma dinâmica (via injeção no construtor ou método setEstrategiaFrete()) e apenas chamar o método calcular(), sem fazer ideia da matemática que está rodando por trás.